Day 117 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 6 reference

Watch: SMOTE Explained with Example (At A Glance!) | Handling data imbalance using SMOTE in python (Statistics and Data science)

`day117_imbalanced_data.py`, segment by segment, in the order `main()` uses things. Metrics and remedies are new; Day 116's softmax regression and Day 107's forest are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: TP, FP, FN, TN = true/false positives/negatives · P, R = precision, recall · n_1, n_0 = rare and common rows · w = a row's weight

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE); sys.path.insert(0, os.path.join(HERE, "..",
    "week-16-classical-ml-trees"))
with contextlib.redirect_stdout(io.StringIO()):
    import day116_stacking_blending as d116
d107, d112, d115 = d116.d107, d116.d112, d116.d115
standardise, stratified_split, d99 = d112.standardise, d112.stratified_split, d112.d99
SPLITS = 20
RATES = [0.02, 0.05, 0.1, 0.2, 0.37]
METHODS = ["none", "class weights", "random oversampling", "SMOTE", "random undersampling",
    "threshold moved"]
```

- `d107, d112, d115 = d116.d107, d116.d112, d116.d115` — earlier lessons reached through Day 116's imports.
- `RATES = [0.02, ..., 0.37]` — 0.37 is breast cancer's own malignant share.

Segment 1 — Metrics

The metrics

MATH

$$AUC = (\Sigma \text{ ranks of rare rows} - n_1(n_1-1)/2) / (n_1 n_0)$$

CODE

```
def confusion(y, pred):
    return int(np.sum((pred == 1) & (y == 1))), int(np.sum((pred == 1) & (y == 0))),
        int(np.sum((pred == 0) & (y == 1))), int(np.sum((pred == 0) & (y == 0)))
def scores(y, p, threshold=0.5):
    tp, fp, fn, tn = confusion(y, (p >= threshold).astype(int))
    prec = tp / max(tp + fp, 1); rec = tp / max(tp + fn, 1)
    return {"accuracy": (tp + tn) / len(y), "precision": prec, "recall": rec, "F1": 2 * prec * rec
        / max(prec + rec, 1e-12),
            "balanced acc": (rec + tn / max(tn + fp, 1)) / 2, "ROC AUC": roc_auc(y, p), "avg
                precision": avg_precision(y, p)}
def roc_auc(y, p):
    r = np.argsort(np.argsort(p, kind="stable"), kind="stable").astype(float)
    for v in np.unique(p):
        m = p == v
        if m.sum() > 1:
            r[m] = r[m].mean()
    n1 = y.sum(); n0 = len(y) - n1
    return float((r[y == 1].sum() - n1 * (n1 - 1) / 2) / (n1 * n0))
def avg_precision(y, p):
    o = np.argsort(-p, kind="stable"); hits = y[o] == 1
    prec = np.cumsum(hits) / np.arange(1, len(y) + 1)
    return float(prec[hits].mean())
```

- `np.argsort(np.argsort(p))` — argsort twice gives each value's rank.
- `r[m] = r[m].mean()` — tied scores share their average rank, so ties count half in the AUC.
- `np.cumsum(hits) / np.arange(1, len(y) + 1)` — precision after each row, down the ranked list.
- `max(tp + fp, 1)` — avoids dividing by zero when nothing is flagged.

Segment 2 — Remedies

smote and resample

MATH

$$x_{new} = x_i + u\,(x_j - x_i), \quad u \sim Uniform(0, 1),\ x_j \text{ one of } x_i\text{'s } k \text{ nearest rare neighbours}$$

CODE

```
def smote(X, k, n_new, rng):
    D = d115.sqdist(X, X); np.fill_diagonal(D, np.inf)
    nn = np.argsort(D, 1)[:, :k]
    i = rng.randint(0, len(X), n_new); j = nn[i, rng.randint(0, min(k, len(X) - 1), n_new)]
    return X[i] + rng.uniform(size=(n_new, 1)) * (X[j] - X[i])
def resample(X, y, method, rng):
    rare, common = np.where(y == 1)[0], np.where(y == 0)[0]; need = len(common) - len(rare)
    if method == "random oversampling":
        extra = rng.choice(rare, need); return np.vstack([X, X[extra]]), np.r_[y, np.ones(need,
            int)]
    if method == "SMOTE":
        return np.vstack([X, smote(X[rare], 5, need, rng)]), np.r_[y, np.ones(need, int)]
    if method == "random undersampling":
        keep = np.r_[rare, rng.choice(common, len(rare), replace=False)]; return X[keep], y[keep]
    return X, y
```

- `np.fill_diagonal(D, np.inf)` — a point is never its own neighbour.
- `rng.uniform(size=(n_new, 1))` — one random fraction per new point; the column shape broadcasts across the features.
- `np.r_[y, np.ones(need, int)]` — np.r_ joins arrays end to end: the old labels, then ones for the new rows.

weighted_logistic and balanced_forest

MATH

$$loss = \Sigma\, w_i \times cross\text{-}entropy_i \,/\, \Sigma\, w_i, \quad w = n \,/\, (2\, n_{class})$$

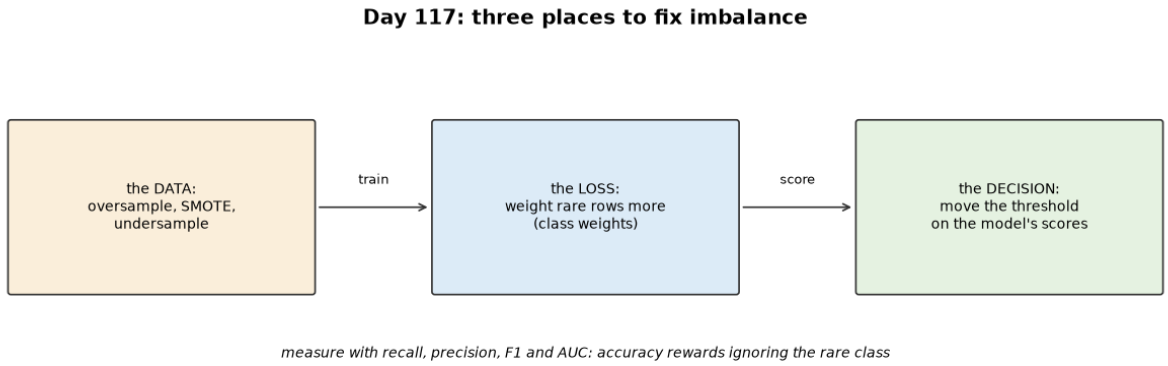
CODE

```
def weighted_logistic(X, y, w, lam=1e-3, steps=500, lr=0.5):
    mu, sd = X.mean(0), X.std(0) + 1e-12; A = (X - mu) / sd; W = np.zeros((A.shape[1], 2)); b =
        np.zeros(2); Y = np.eye(2)[y]
    m = [np.zeros_like(W), np.zeros_like(b)]; v = [np.zeros_like(W), np.zeros_like(b)]
    for t in range(1, steps + 1):
        G = w[:, None] * (d116.softmax(A @ W + b) - Y) / w.sum()
        for i, (p, g) in enumerate(((W, A.T @ G + lam * W), (b, G.sum(0)))):
            m[i] = 0.9 * m[i] + 0.1 * g; v[i] = 0.999 * v[i] + 0.001 * g * g
            p -= lr * (m[i] / (1 - 0.9 ** t)) / (np.sqrt(v[i] / (1 - 0.999 ** t)) + 1e-8)
    return lambda Z: d116.softmax(((Z - mu) / sd) @ W + b)[:, 1]
def balanced_forest(X, y, seed, trees=60):
    rng = np.random.RandomState(seed); rare, common = np.where(y == 1)[0], np.where(y == 0)[0]; m
        = max(1, int(np.sqrt(X.shape[1]))); fs = []
    for _ in range(trees):
        idx = np.r_[rng.choice(rare, len(rare)), rng.choice(common, len(rare))]
        fs.append(d107.grow_random(X[idx], y[idx], 2, m, rng))
    return lambda Z: d107.forest_proba(fs, Z, 2)[:, 1]
```

- `w[:, None] * (...) / w.sum()` — each row's gradient scaled by its weight; dividing by the total keeps the step size comparable.
- `np.r_[rng.choice(rare, len(rare)), rng.choice(common, len(rare))]` — a bootstrap with as many common rows as rare ones.

- `d107.grow_random(X[idx], y[idx], 2, m, rng)` — Day 107's tree, unchanged, on the balanced sample.

SYSTEM DIAGRAM



Three places to fix imbalance.

Segment 3 — The experiments

fit and best_threshold

CODE

```
def fit(model, method, X, y, seed):
    rng = np.random.RandomState(seed)
    if method == "threshold moved":
        method = "none"
    if method == "class weights":
        if model == "logistic":
            w = np.where(y == 1, len(y) / (2 * y.sum()), len(y) / (2 * (len(y) - y.sum())))
            return weighted_logistic(X, y, w)
        return balanced_forest(X, y, seed)
    A, b = resample(X, y, method, rng)
    if model == "logistic":
        return weighted_logistic(A, b, np.ones(len(b)))
    f, _ = d107.fit_forest(A, b, 2, 60, max(1, int(np.sqrt(X.shape[1]))), seed)
    return lambda Z: d107.forest_proba(f, Z, 2)[:, 1]
def best_threshold(y, p):
    cand = np.unique(p); f = [scores(y, p, t)["F1"] for t in cand]
    return float(cand[int(np.argmax(f))])
```

- `if method == "threshold moved": method = "none"` — the model is trained as usual; only the decision changes, later.
- `cand = np.unique(p)` — every distinct score is a possible threshold.

The threshold, inside demo_compare

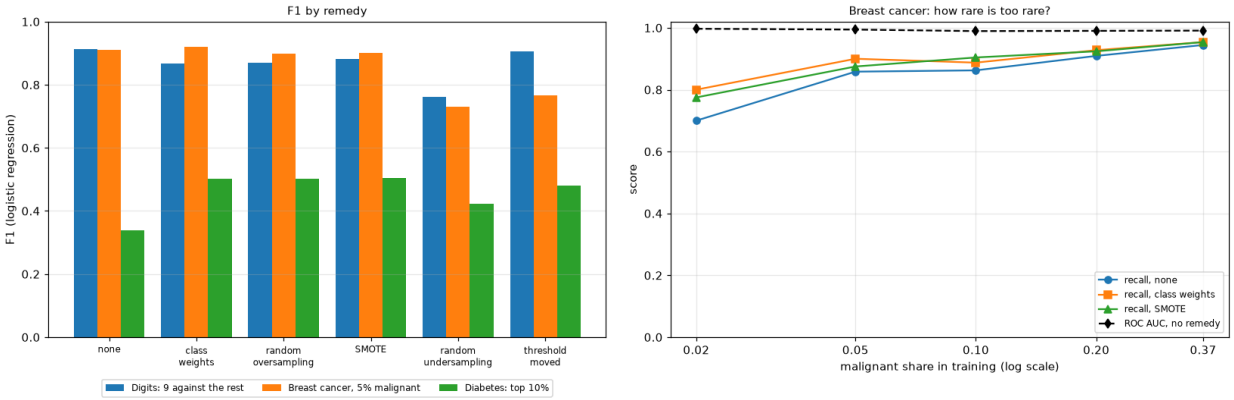
CODE

```
if m == "threshold moved":
    o = np.random.RandomState(s).permutation(len(y)); v = len(y) // 4
    t = best_threshold(y[o[:v]], fit(model, m, X[o[v:]], y[o[v:]], s)(X[o[:v]])) if y[o[:v]].sum()
        else 0.5
    R[m].append(scores(yt, fit(model, m, X, y, s)(Xt), t))
```

- `o[:v], o[v:]` — a quarter of the training rows chooses the threshold; the rest train the model for that choice.

- `if y[o[:v]].sum() else 0.5` — with no rare row in that quarter, no threshold can be chosen: keep 0.5.

REAL GRAPH



Real graph from the script: F1 by remedy, and how rare is too rare.

Quick Syntax Glossary — new or reinforced constructs

- `np.argsort(np.argsort(p))` — ranks.
- `np.fill_diagonal(D, v)` — set the diagonal.
- `np.r_[a, b]` — join arrays end to end.
- `w[:, None] * G` — scale each row by its weight.
- `{k: f(v) for k, v in d.items()}` — a dictionary comprehension: the mean of every metric at once.
- `f(..., t)` — a threshold passed as an argument.

End of Day 117 syntax reference. For the experiments and what they showed, see the Day 117 complete guide.